Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

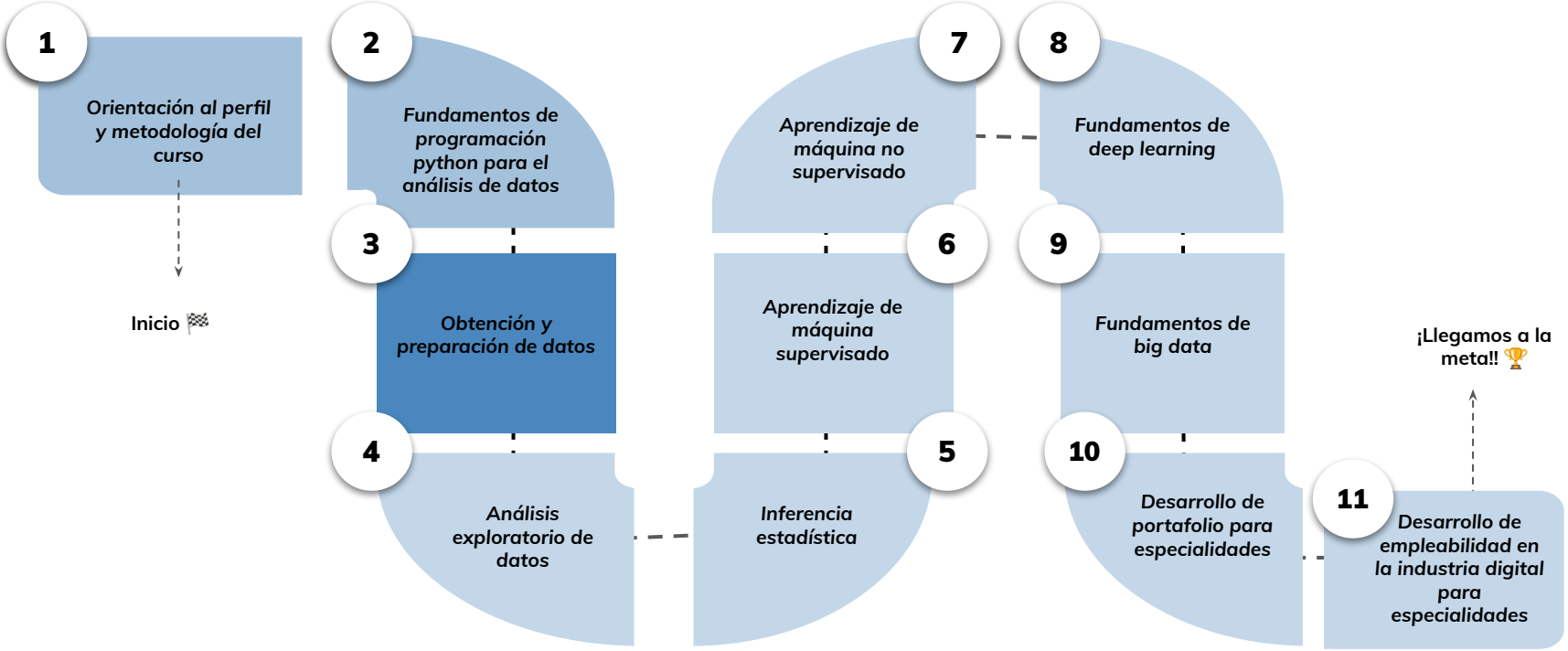
Clase grabada

› Obtención de datos desde archivos - Parte I

Aprendizaje Esperado 3: Aplicar técnicas de extracción de datos desde distintas fuentes utilizando librerías utilitarias de python para su posterior uso.

Hoja de ruta

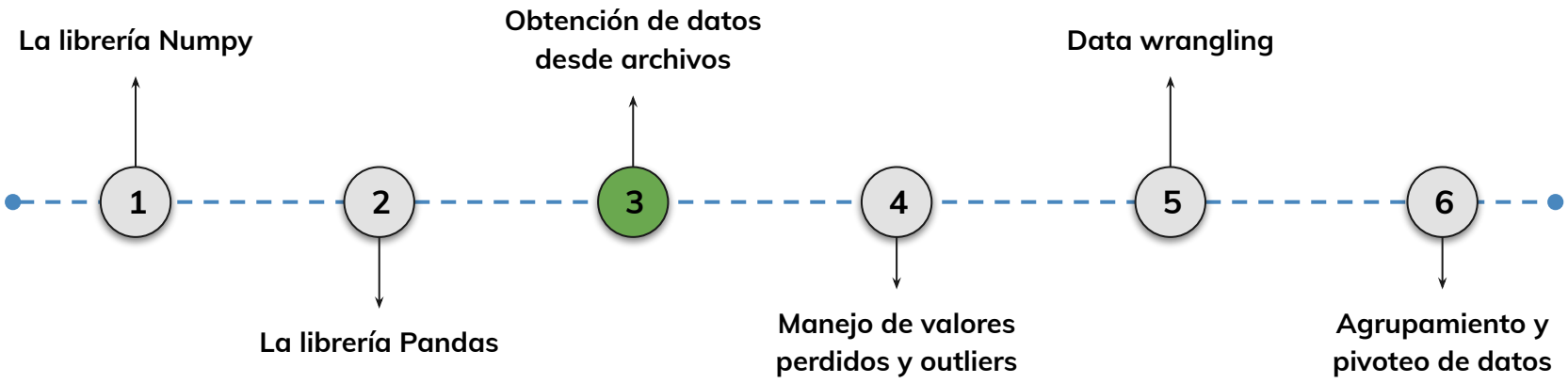
¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



alkemy

Roadmap de lecciones

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



alkemy

Learning Path

¿Cuáles temas trabajaremos hoy?

3.

Obtención de datos desde archivos en Python

Aprenderemos a extraer datos desde fuentes estructuradas como CSV, Excel y Web, utilizando librerías de Python como pandas, y a almacenarlos para su posterior análisis.

- **Lectura y escritura de archivos CSV**
  - **read_csv() y to_csv()**
  - **Parámetros útiles (sep, header, etc.)**
- **Lectura y escritura de archivos Excel**
  - **read_excel() y to_excel()**
  - **Uso de sheet_name, múltiples hojas, ExcelWriter()**

alkemy

Objetivos de aprendizaje

¿Qué aprenderás?

- Aprender a leer archivos CSV desde distintas configuraciones.
- Aprender a escribir datos en archivos CSV.
- Dominar el uso de pandas para manipular archivos Excel.
- Extraer tablas directamente desde sitios web con read_html().
- Aplicar técnicas de limpieza y estructuración posterior de datos obtenidos.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos 📚:

→ *Aplicamos métodos de selección de datos con .loc[] y .iloc[]*

→ *Filtramos registros usando condiciones simples y compuestas*

→ *Exploramos datasets utilizando head(), tail(), info() y describe()*

→ *Resumimos columnas con funciones estadísticas y analizamos frecuencias con value_counts()*

Obtención de datos desde archivos

Obtención de datos desde archivos

En el análisis de datos, la obtención y manipulación eficiente de la información es un paso fundamental en cualquier flujo de trabajo de ciencia de datos, ingeniería de datos y análisis empresarial. La calidad de los datos impacta directamente en la precisión de los resultados obtenidos en modelos estadísticos, visualización de datos y aprendizaje automático.

Introducción

Python se ha convertido en un lenguaje estándar para la manipulación de datos, y una de sus bibliotecas más potentes es **Pandas**, la cual ofrece métodos sencillos y eficientes para la lectura y escritura de archivos en múltiples formatos.

Importancia de la obtención de datos

Fundamento del análisis

Contar con herramientas que permitan extraer, limpiar y estructurar datos de diversas fuentes de manera eficiente es crucial para cualquier profesional del área.

Impacto en resultados

La calidad de los datos impacta directamente en la precisión de los resultados obtenidos en modelos estadísticos, visualización de datos y aprendizaje automático.

Eficiencia del proceso

La obtención y manipulación eficiente de la información es un paso fundamental en cualquier flujo de trabajo de ciencia de datos, ingeniería de datos y análisis empresarial.

Obtención de datos desde archivos csv

Los archivos CSV (Comma-Separated Values) son ampliamente utilizados en la ciencia de datos y en el almacenamiento de datos estructurados debido a su simplicidad y compatibilidad con diversas plataformas.

Características de los archivos CSV

Estructura simple

Estos archivos contienen información organizada en filas y columnas, separadas por comas, punto y coma u otros delimitadores.

Alta compatibilidad

Son compatibles con diversas plataformas y programas, lo que facilita el intercambio de datos entre diferentes sistemas.

Almacenamiento eficiente

Permiten almacenar grandes volúmenes de datos estructurados de manera eficiente y con un formato ligero.

Ventajas de Pandas para archivos CSV

Pandas proporciona una serie de herramientas que facilitan la lectura y escritura de archivos CSV, permitiendo a los usuarios cargar, analizar y modificar grandes volúmenes de datos con facilidad.

Además, Pandas permite manejar diferentes codificaciones, delimitadores y formatos de archivo sin comprometer la integridad de los datos.

Leyendo un archivo CSV

El método read_csv() de Pandas es la forma más eficiente de leer archivos CSV y convertirlos en un DataFrame, la estructura de datos tabular de Pandas.

```
import pandas as pd

# Leer un archivo CSV básico
df = pd.read_csv('datos.csv')

# Mostrar las primeras filas
print(df.head())
```

Opciones para leer archivos CSV

En ocasiones, los archivos CSV utilizan delimitadores distintos a la coma. Para estos casos, se puede especificar el delimitador correcto mediante el parámetro sep:

```
# Leer un archivo CSV con punto y coma como delimitador
df = pd.read_csv('datos.csv', sep=';')

# Leer un archivo CSV con tabulación como delimitador
df = pd.read_csv('datos.csv', sep='\t')
```

Parámetros adicionales para read_csv()

1

header=None

Indica que el archivo no tiene encabezados, lo que permite asignar nombres de columnas manualmente.

```
df = pd.read_csv('datos.csv',
header=None)
```

2

names=['Col1', 'Col2']

Permite asignar nombres personalizados a las columnas.

```
df = pd.read_csv('datos.csv',
names=['Nombre', 'Edad',
'Ciudad'])
```

3

na_values=['?', 'N/A']

Especifica valores que deben considerarse como nulos.

```
df = pd.read_csv('datos.csv',
na_values=['?', 'N/A', 'NA'])
```

Escribiendo un archivo CSV

Para guardar un DataFrame en un archivo CSV, se utiliza el método to_csv().

```
# Guardar un DataFrame en un archivo CSV
df.to_csv('nuevo_datos.csv')

# Guardar sin incluir el índice
df.to_csv('nuevo_datos.csv', index=False)
```

Opciones para escribir archivos CSV

Delimitador personalizado

sep=';': Define un delimitador diferente.

```
df.to_csv('nuevo_datos.csv', sep=';')
```

Codificación específica

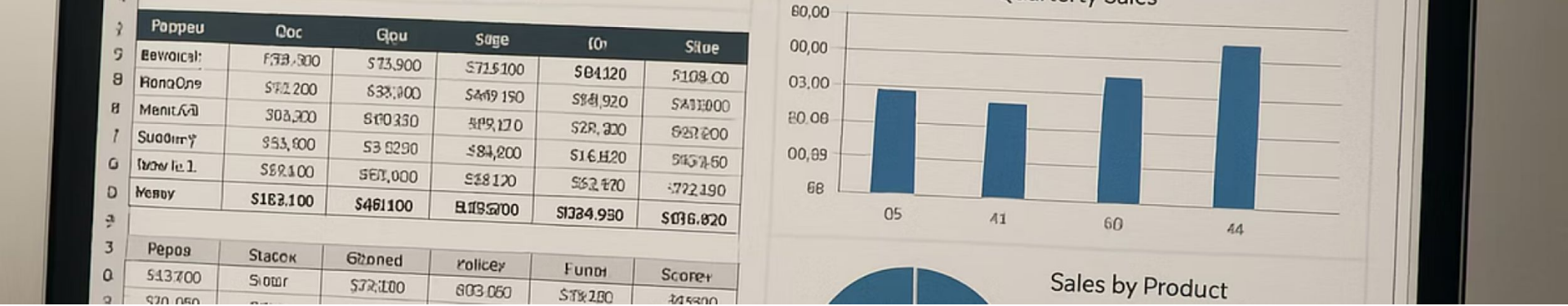
encoding='utf-8': Especifica la codificación del archivo para evitar problemas con caracteres especiales.

```
df.to_csv('nuevo_datos.csv', encoding='utf-8')
```

Sin encabezados

header=False: Omite los nombres de las columnas.

```
df.to_csv('nuevo_datos.csv', header=False)
```



ARCHIVOS EXCEL

Los archivos Excel son ampliamente utilizados en análisis de datos, finanzas y reportes empresariales debido a su capacidad de organizar información en múltiples hojas de cálculo.

Ventajas de los archivos Excel

Múltiples hojas

Permiten organizar información en múltiples hojas de cálculo dentro de un mismo archivo.

Fórmulas integradas

Ofrecen capacidades de cálculo y análisis mediante fórmulas y funciones integradas.

Visualización de datos

Facilitan la creación de gráficos y visualizaciones directamente desde los datos.

Pandas y archivos Excel

Pandas permite leer y escribir archivos Excel con gran facilidad, integrando librerías externas para manejar distintos formatos.

Esta integración facilita el trabajo con archivos Excel en flujos de análisis de datos, permitiendo una transición fluida entre diferentes formatos.

Librería XLRD

xlrd fue tradicionalmente utilizada para leer archivos Excel en Pandas. Sin embargo, debido a cambios recientes en Pandas, ahora se recomienda usar openpyxl para manejar archivos .xlsx modernos.

Para garantizar compatibilidad con archivos Excel, se recomienda instalar openpyxl si aún no está disponible:

```
# Instalar openpyxl para manejar archivos Excel

pip install openpyxl
```

Leyendo un archivo Excel

Para cargar datos desde un archivo Excel en un DataFrame, se usa el método read_excel().

```
import pandas as pd

# Leer un archivo Excel
df = pd.read_excel('datos.xlsx')

# Mostrar las primeras filas
print(df.head())
```

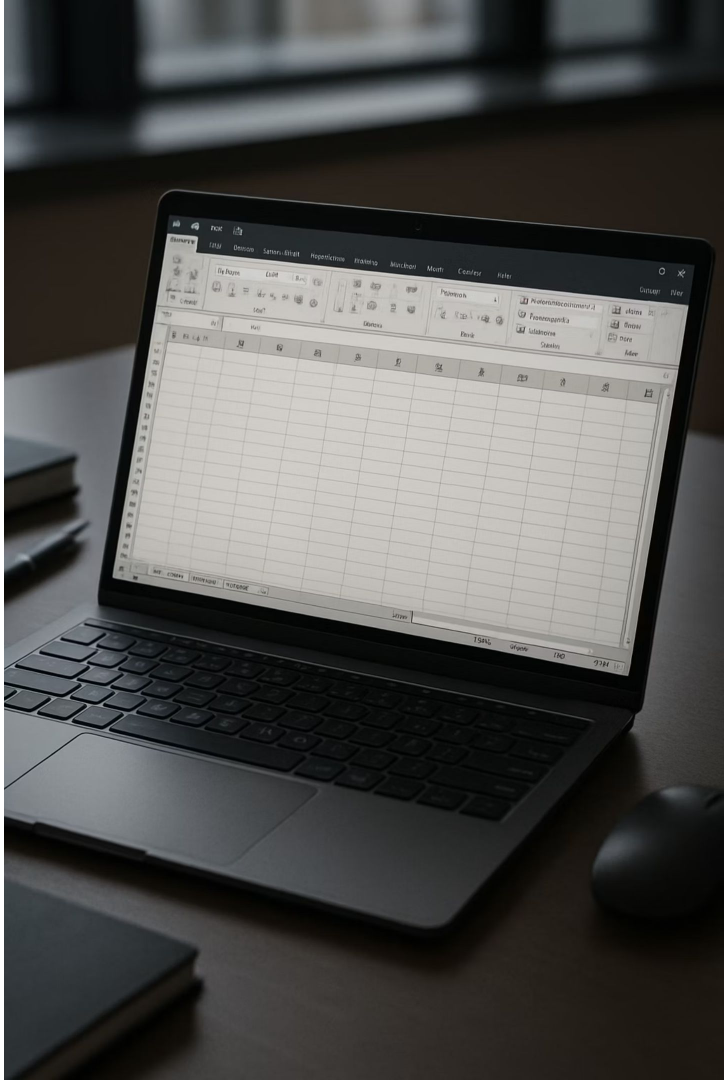
Leyendo hojas específicas de Excel

Si el archivo contiene varias hojas, se puede especificar la hoja deseada:

```
# Leer una hoja específica por nombre
df = pd.read_excel('datos.xlsx', sheet_name='Ventas')

# Leer una hoja específica por índice (0 es la primera
hoja)
df = pd.read_excel('datos.xlsx', sheet_name=0)

# Leer múltiples hojas (devuelve un diccionario de
DataFrames)
dfs = pd.read_excel('datos.xlsx', sheet_name=['Ventas',
'Gastos'])
```



Escribiendo un archivo Excel

Para exportar datos en formato Excel, se usa to_excel().

```
# Guardar un DataFrame en un archivo Excel
df.to_excel('nuevo_datos.xlsx')


# Guardar sin incluir el índice
df.to_excel('nuevo_datos.xlsx', index=False)
```

Opciones avanzadas para escribir en Excel

Múltiples hojas

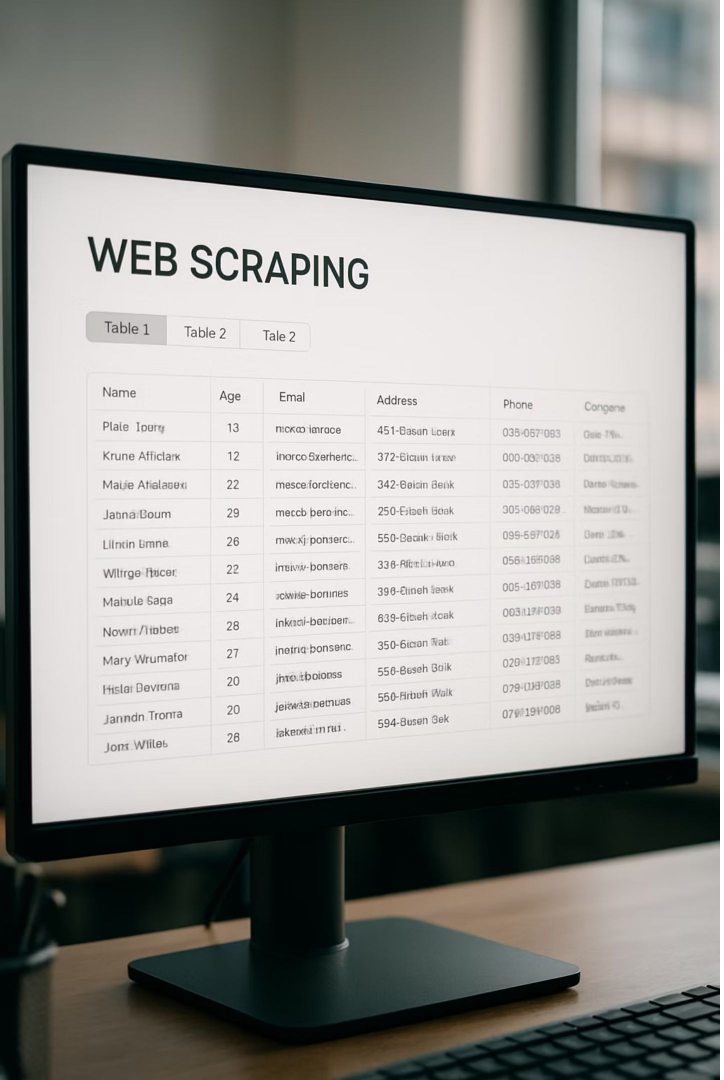
Es posible escribir diferentes DataFrames en distintas hojas del mismo archivo Excel.

```
with pd.ExcelWriter('nuevo_datos.xlsx') as
writer:
    df1.to_excel(writer,
sheet_name='Ventas')
    df2.to_excel(writer,
sheet_name='Gastos')
```

Formato personalizado

Se pueden aplicar formatos específicos a las celdas utilizando librerías complementarias como XlsxWriter.

```
df.to_excel('nuevo_datos.xlsx',
engine='xlsxwriter')
```



Leer tablas web con pandas

El método read_html() de Pandas permite extraer datos estructurados de sitios web que contienen tablas en formato HTML.

Extracción de tablas web

La capacidad de extraer tablas directamente desde páginas web facilita el acceso a información actualizada sin necesidad de descargar archivos manualmente.

Esta funcionalidad es especialmente útil para análisis de datos que requieren información actualizada regularmente, como precios, estadísticas o resultados deportivos.

```
# Leer una tabla desde Wikipedia
url = 'https://es.wikipedia.org/wiki/Lista_de_pa%C3%ADses_por_PIB_(nominal)'
df_list = pd.read_html(url)

# Mostrar la primera tabla encontrada
df = df_list[0]
print(df.head())
```

Uso de read_html()

```
import pandas as pd

# Extraer todas las tablas de una URL
url = 'https://en.wikipedia.org/wiki/List_of_countries_by_population'
tablas = pd.read_html(url)

# La función devuelve una lista de DataFrames, uno por cada tabla encontrada
print(f"Número de tablas encontradas: {len(tablas)}")

# Acceder a la primera tabla
df = tablas[0]
print(df.head())
```

Opciones para read_html()

1

Filtrado por atributos

Es posible filtrar tablas específicas utilizando atributos HTML como id o class.

```
# Extraer tabla con un id específico
tablas = pd.read_html(url, attrs={'id': 'tabla-datos'})
```

2

Selección por índice

Se puede especificar qué tabla extraer mediante su índice en la página.

```
# Extraer solo la segunda tabla de la página
tablas = pd.read_html(url, match='Población')
```

3

Filtrado por texto

Es posible filtrar tablas que contengan un texto específico.

```
# Extraer tablas que contengan la palabra "Población"
tablas = pd.read_html(url, match='Población')
```

Requisitos para read_html()

Dependencias

Para utilizar read_html() es necesario tener instaladas las bibliotecas lxml, html5lib y BeautifulSoup4.

Instalación

Se pueden instalar con pip: pip install lxml html5lib beautifulsoup4

Limitaciones

Solo funciona con tablas HTML bien formadas. Sitios con protección contra scraping pueden bloquear las solicitudes.

Procesamiento posterior de tablas web

Después de extraer tablas web, generalmente es necesario realizar algún procesamiento para limpiar y estructurar los datos correctamente.

- Eliminar filas o columnas innecesarias
- Convertir tipos de datos
- Renombrar columnas
- Manejar valores faltantes

```
# Ejemplo de procesamiento posterior
df = tablas[0]

# Renombrar columnas
df.columns = ['País', 'Población', 'Fecha',
'Fuente']

# Convertir población a tipo numérico
df['Población'] =
df['Población'].str.replace(',',
'').astype(int)

# Eliminar filas con datos faltantes
df = df.dropna()
```

Comparación de métodos de obtención de datos

1

Archivos CSV

- Formato simple y ligero
- Ampliamente compatible
- Ideal para datos tabulares simples
- Limitado a una sola "hoja" de datos

2

Archivos Excel

- Soporte para múltiples hojas
- Capacidades de formato avanzadas
- Integración con fórmulas y gráficos
- Archivos más pesados

3

Tablas Web

- Acceso a datos actualizados
- No requiere descarga manual
- Automatizable para actualizaciones
- Dependiente de la estructura del sitio web

Flujo de trabajo para la obtención de datos

Identificar fuente

Determinar el formato y ubicación de los datos necesarios (CSV, Excel, Web)

Obtener datos

Utilizar el método apropiado de Pandas (read_csv, read_excel, read_html)

Limpiar datos

Procesar y estructurar los datos obtenidos (tipos, valores nulos, etc.)

Analizar datos

Realizar el análisis requerido con los datos ya estructurados

Manejo de errores comunes

Problemas de codificación

Los archivos CSV pueden tener diferentes codificaciones que causan problemas con caracteres especiales.

```
# Solución: especificar la codificación correcta
df = pd.read_csv('datos.csv', encoding='latin1')# odf = pd.read_csv('datos.csv', encoding='utf-8')
```

Delimitadores incorrectos

Algunos archivos CSV utilizan delimitadores distintos a la coma estándar.

```
# Solución: especificar el delimitador correcto
df = pd.read_csv('datos.csv', sep=';')
```

Hojas de Excel no encontradas

Intentar acceder a una hoja que no existe en el archivo Excel.

```
# Solución: listar las hojas disponibles primero
xls = pd.ExcelFile('datos.xlsx')print(xls.sheet_names)
```

Live Coding

¿En qué consistirá la Demo?

Vamos a desarrollar un mini flujo de trabajo que obtenga una tabla desde una página web usando pandas.read_html(), la limpie y la guarde como CSV para su posterior análisis.

1. *Importar las librerías necesarias (pandas)*
2. *Elegir una página web con tablas HTML públicas (ej: Wikipedia)*
3. *Usar read_html() para extraer todas las tablas*
4. *Seleccionar una tabla por índice*
5. *Renombrar columnas de forma explícita*
6. *Convertir tipos de datos (ej: población con .str.replace().astype())*
   Eliminar filas con valores faltantes
7. *Guardar el resultado en un archivo CSV con to_csv()*

Tiempo: *20 minutos*

#Momentode Preguntas...

¿?

Qué parámetro permite cambiar el delimitador en read_csv()?

¿?

¿Cómo se accede a una hoja específica en un Excel con pandas?

¿?

¿Qué problemas pueden surgir al leer archivos con codificación especial?

¿?

¿Qué librerías adicionales se requieren para read_html()?

Momento:

Time-out!

⏰5 -10 min.

alkemy

Ejercicio N° 1

¡Extrae, transforma y guarda!

¡Extrae, transforma y guarda!

Contexto: 🙌

Estás trabajando como analista de datos junior en una empresa de retail. Tu tarea es automatizar el procesamiento de archivos que contienen las ventas semanales en formato Excel y CSV, y consolidar la información para análisis posteriores.

Consigna: ✍️

Utilizando las librerías pandas y openpyxl, deberás:

- Leer un archivo .csv con separador ;.
- Leer un archivo .xlsx que contiene múltiples hojas.
- Unificar ambos DataFrames y eliminar columnas innecesarias.
- Guardar el resultado como un nuevo archivo .csv sin índice.

Tiempo 🕐**:** *45 minutos*

Paso a paso: ⚙️

1. Crea un entorno en Google Colab
2. Importa la librería pandas.
3. Usa read_csv('ventas.csv', sep=';') para cargar el CSV.
4. Usa read_excel('reporte.xlsx', sheet_name=None) para cargar todas las hojas.
5. Combina los datos relevantes en un único DataFrame.
6. Elimina columnas innecesarias con drop().
7. Guarda con to_csv('consolidado.csv', index=False).

alkemy

¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ ***Conocer los métodos de lectura de archivos CSV con pandas***
- ✓ ***Manipular archivos Excel con múltiples hojas***
- ✓ ***Leer tablas desde páginas web***
- ✓ ***Utilizar parámetros para controlar encabezados, codificación y delimitadores***
- ✓ ***Exportar resultados en archivos nuevos***

¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las ***actividades disponibles en la plataforma.*** *Estas propuestas son clave para afianzar lo trabajado y* ***forman parte obligatoria del recorrido de aprendizaje.***

💪 Análisis de caso ——— 💪 Selección Múltiple

💪 Comprensión lectora

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

alkemy

< ¡Muchas gracias! >

alkemy